

DJI-Market.ru

# Sales Performance Dashboard

Middle Full-Stack Developer / AI-Native Developer

|                    |                        |            |                |
|--------------------|------------------------|------------|----------------|
| Frontend           | Backend                | Data       | Run            |
| React + TypeScript | ASP.NET Core + EF Core | PostgreSQL | Docker Compose |

**Задача в одном предложении**

За 8 часов собрать рабочий, визуально качественный Full-Stack dashboard аналитики продаж менеджеров, который поднимается локально одной командой и демонстрирует не только код, но и инженерный процесс работы с AI.

|                     |                      |                                     |
|---------------------|----------------------|-------------------------------------|
| Timebox<br>8 часов  | Формат<br>Full-Stack | Авторизация<br>не требуется         |
| AI<br>можно и нужно | UI<br>desktop        | Запуск<br>docker compose up --build |

Главный принцип: AI-generated код нормален. Код, который разработчик не понимает и не может объяснить, — нет.

# 1. Контекст и ожидаемый результат

Это не тест на синтаксис и не задача «сверстать несколько карточек».

Нужно разработать небольшое самостоятельное Full-Stack приложение для анализа продаж менеджеров. Нам интересно посмотреть, как вы целиком решаете продуктовую задачу:

- проектируете модель данных;
- строите backend и API;
- организуете работу с PostgreSQL;
- собираете современный React-интерфейс;
- обрабатываете ошибки и пустые состояния;
- пишете тесты;
- используете AI и проверяете его результат;
- доводите всё до запуска одной командой.

## Результат

После клонирования проекта мы должны выполнить `docker compose up --build`, дождаться старта сервисов, открыть браузер и сразу увидеть заполненный рабочий dashboard.

# 2. Обязательный стек

## Backend

C# • .NET 8+ • ASP.NET Core • Entity Framework Core • REST API • async/await

## Frontend

React • TypeScript. Остальные библиотеки выбираются самостоятельно.

## Data

PostgreSQL. Миграции обязательны. SQL можно использовать напрямую, если это оправдано.

## Infrastructure

Docker • Docker Compose. Приложение должно подниматься одной командой.

Дополнительные библиотеки разрешены, если они помогают решать задачу, а не просто усложняют проект.

# 3. Модель данных

Сделайте предметную область достаточно реалистичной, чтобы dashboard выглядел как часть настоящего B2B-продукта.

## Manager

Менеджер отдела продаж: имя, команда/должность, активность, данные для avatar/initials.

## Customer

Клиент: имя, компания, сегмент.

## Category

Категория товаров.

## Product

Товар: название, категория и необходимые атрибуты.

**Sale**

Продажа: менеджер, клиент, дата, статус.

**SaleItem**

Позиция продажи: товар, количество, цена продажи и себестоимость.

**Минимальные статусы продажи:**

- Paid
- Cancelled
- Refunded

**Минимальные вычисляемые показатели:**

|               |                             |
|---------------|-----------------------------|
| Revenue       | сумма оплаченных продаж     |
| Gross Profit  | Revenue – Cost              |
| Margin        | Gross Profit / Revenue      |
| Average Check | Revenue / количество продаж |

**Важно**

Refunded можно трактовать по-разному. Выберите разумное правило, применяйте его последовательно во всей системе и опишите в README. Нам важнее логика решения, чем совпадение с нашей трактовкой.

## 4. Seed-данные

База должна автоматически наполняться после первого запуска.

| Набор     | Ориентир                    |
|-----------|-----------------------------|
| Менеджеры | 15–25                       |
| Клиенты   | 50–100                      |
| Категории | несколько                   |
| Товары    | несколько десятков          |
| Продажи   | 2 000–5 000                 |
| Период    | минимум 6 месяцев, лучше 12 |

**Данные должны быть неравномерными и похожими на реальные:**

- сильные и слабые менеджеры;
- разные средние чеки;
- разная маржинальность;
- сезонность;
- отмены и возвраты;
- крупные и мелкие сделки;
- периоды без продаж у отдельных менеджеров.

Seed желательно сделать воспроизводимым: после пересоздания БД должны получаться одинаковые данные.

### AI приветствуется

Можно использовать AI для генерации сценариев, названий, распределений и самого seed-кода.

## 5. Dashboard

Нужна законченная desktop-страница современного B2B/SaaS продукта, а не набор демо-компонентов.

Целевой viewport для проверки: примерно 1440×900. Mobile-версия не требуется.

|                                                                                                                                                                                |                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>KPI-карточки</b><br><br>Выручка, валовая прибыль, маржинальность, количество продаж, средний чек, лучший менеджер. Желательно сравнение с предыдущим сопоставимым периодом. | <b>Период</b><br><br>Сегодня, 7 дней, 30 дней, этот месяц, прошлый месяц и произвольный диапазон from → to. |
| <b>Рейтинг менеджеров</b><br><br>Переключение минимум между Gross Profit и Average Check.                                                                                      | <b>Динамика</b><br><br>График Revenue / Gross Profit / количества продаж во времени.                        |
| <b>Категории и продукты</b><br><br>Продажи/прибыль по категориям и лучшие продукты.                                                                                            | <b>Последние продажи</b><br><br>Дата, менеджер, клиент, товары, статус, сумма, валовая прибыль.             |

**Дополнительные полезные блоки приветствуются:**

- сравнение менеджеров;
- sparklines;
- scatter plot или bar charts;
- распределение продаж и маржи;
- доля категорий;
- любая аналитика, которую руководителю продаж реально было бы полезно видеть.

**Продуктовая инициатива**

Перечень аналитики не закрытый. Если вы видите полезный блок, который помогает руководителю быстрее понять картину продаж, добавляйте.

## 6. Рейтинг менеджеров

Один из центральных элементов страницы.

**Минимум два режима ранжирования:**

- по валовой прибыли (Gross Profit)
- по среднему чеку (Average Check)

**В строке/карточке менеджера желательно показывать:**

- позицию в рейтинге
- менеджера
- количество продаж
- выручку
- валовую прибыль
- средний чек
- маржинальность
- изменение к предыдущему периоду

## 7. Backend и API

Основные вычисления и фильтрация должны выполняться на сервере.

### Не делайте так

Frontend не должен загружать несколько тысяч продаж и самостоятельно пересчитывать весь dashboard в браузере.

### Хотим увидеть:

- понятные REST API-контракты;
- разумное разделение ответственности;
- эффективную работу с PostgreSQL;
- отсутствие очевидных N+1;
- async I/O;
- валидацию;
- понятную обработку ошибок;
- расширяемую, но не переусложнённую структуру кода.

### Архитектура

Clean Architecture, CQRS, Repository и другие паттерны не обязательны. Если простой модуль можно сделать просто — делайте просто. Дополнительный слой должен решать конкретную проблему.

## 8. PostgreSQL

Мы отдельно смотрим на работу с данными, а не только на умение пользоваться ORM.

- EF Core migrations;
- разумные связи и типы данных;
- индексы там, где они действительно нужны;
- корректные aggregate queries;
- понимание, какой SQL генерирует ORM;
- возможность использовать raw SQL / Dapper для отдельной аналитической выборки, если это оправдано.

Главное — уметь объяснить, почему выбран именно такой способ.

## 9. Frontend и UX

Визуальное качество — полноценная часть оценки.

### Интерфейс должен быть:

- современным
- визуально аккуратным
- информационно насыщенным
- легко читаемым
- отзывчивым
- с понятной иерархией

### Можно использовать любые разумные библиотеки:

- TanStack Query
- Zustand / Redux Toolkit
- React Hook Form
- Tailwind / MUI / Ant Design / shadcn
- Framer Motion
- любую библиотеку графиков

Можно искать референсы, брать идеи у современных SaaS/FinTech/CRM dashboard и использовать AI для генерации визуальной концепции.

## 10. Анимации и состояния интерфейса

Dashboard должен ощущаться живым, но не превращаться в презентацию.

### Примеры уместных микроанимаций:

- появление KPI-карточек
- animated counters
- переходы графиков
- hover states
- плавное перестроение рейтинга
- смена периода
- skeleton loading
- transitions таблиц
- feedback на действия

### Обязательные состояния:

- initial loading
- loading при смене периода
- ошибка API
- пустой период без продаж
- отсутствие данных по отдельному блоку

#### Критерий

Вечный spinner, белый экран или внезапно пустой блок без объяснения — это не production-ready UX.

## 11. Edge cases

Мы будем менять периоды и смотреть, корректно ли ведёт себя система.

- Cancelled-продажи;
- Refunded-продажи;
- менеджер без продаж за период;
- одинаковые результаты двух менеджеров;
- продажа ровно на границе диапазона дат;
- одна очень крупная продажа;
- большое количество мелких продаж;
- разные себестоимости и маржа.

## 12. Тесты

Не нужно гнаться за coverage. Проверьте то, что реально может сломать бизнес-результат.

| Backend                   | Frontend              |
|---------------------------|-----------------------|
| расчёт KPI                | смена периода         |
| фильтр периода            | переключение рейтинга |
| рейтинг                   | loading/error state   |
| обработка статусов продаж |                       |

## 13. Авторизация

Не тратьте время на функциональность, которую мы не проверяем.

### Вне scope

Login, JWT, регистрация, OAuth, управление пользователями и permissions реализовывать не нужно. Считайте, что пользователь уже вошёл и имеет право видеть dashboard.

## 14. Docker и локальный запуск

Это обязательное требование.

После клонирования репозитория мы выполняем:

```
docker compose up --build
```

После этого должны автоматически:

- подняться frontend, backend и PostgreSQL;
- примениться migrations;
- создаться seed-данные;
- стать доступным dashboard в браузере.

### Недостаток решения

Если для запуска нужно вручную открывать PgAdmin, выполнять SQL, запускать seed отдельной командой или отдельно устанавливать зависимости — это будет считаться минусом.

## 15. AI — обязательная часть процесса

Нас интересует не процент кода, написанный человеком вручную, а качество инженерной работы с агентом.

### Можно использовать:

- Claude Code
- Codex
- Cursor
- Gemini
- Copilot
- OpenCode
- другие coding agents и модели

### Мы будем смотреть:

- как вы ставите задачу модели;
- какой контекст даёте;
- как декомпозируете работу;
- что делегируете;
- что проверяете вручную;
- как реагируете на ошибки модели;
- можете ли объяснить итоговый код.

### Правило

Можно отдать AI половину реализации. Можно 90%. На следующем этапе мы можем открыть практически любой участок решения и спросить: «Почему здесь сделано именно так?» — и вы должны быть способны ответить.

## 16. AI\_PROMPTS.md

Обязательный журнал ваших запросов к AI.

С самого начала работы создайте файл:

```
AI_PROMPTS.md
```

В него последовательно сохраняйте все свои запросы и инструкции к AI/coding-agent. Желательный формат:

```
## 14:32 — Claude Code / Opus

<ваш запрос модели дословно>
```

**Нам нужны именно сообщения, которые вы сами отправляли AI.**

- не нужно сохранять скрытый chain-of-thought модели;
- не нужно сохранять внутренние tool traces;
- не нужно сохранять служебные сообщения провайдера;
- не записывайте API keys, токены, пароли и другие секреты.

Если инструмент позволяет, можно автоматизировать сохранение prompt history. Если нет — допустим экспорт или ручной перенос пользовательских сообщений из истории сессии.

## 17. AI\_NOTES.md

Короткая рефлексия по работе с AI.

```
AI_NOTES.md
```

Достаточно 10–30 содержательных строк:

- какими моделями и инструментами пользовались;
- какие части задачи делегировали;
- что проектировали самостоятельно;
- где AI заметно ускорил работу;
- где AI ошибся;
- какое предложение агента изменили или отклонили;
- как проверяли итоговый код.

## 18. README.md

README должен позволить понять и запустить проект без переписки с автором.

**Обязательно включите:**

- точную инструкцию запуска;
- краткое описание frontend / backend / БД / API;
- business rules: Revenue, Gross Profit, Refund, Cancelled и предыдущий период;
- основные технические решения и индексы;
- что не успели за 8 часов;
- что улучшили бы дальше в production.

## 19. Git

История коммитов — часть инженерного процесса.

Не нужно делать 50 коммитов, но один финальный commit на весь проект менее информативен, чем несколько осмысленных этапов.

```
initial project structure
add sales domain and seed
implement analytics API
build dashboard UI
add charts and animations
add tests
```

## 20. Что специально НЕ требуется

- авторизация и регистрация;
- управление пользователями;
- mobile layout;
- отдельная admin panel;
- Kubernetes;
- production cloud deployment;
- сложная distributed architecture;
- микросервисы ради микросервисов.

### Принцип

Это маленькая система. Мы не даём дополнительных баллов за архитектуру на 40 проектов, если задача нормально решается проще.

## 21. Timebox

Ориентир — 8 часов фактической работы.

Не нужно после восьми часов ещё несколько вечеров доводить dashboard до идеала. Нам интересно посмотреть, как вы расставляете приоритеты и что считаете самым важным.

### Не успели — нормально

Оставьте систему рабочей и честно опишите в README, что именно не успели и что сделали бы дальше. Это лучше, чем сломанный проект с большим количеством незавершённых идей.

## 22. Что сдавать

Нужна ссылка на GitHub/GitLab или архив проекта.

**Ожидаем примерно такую структуру:**

```
/frontend
/backend
docker-compose.yml
README.md
AI_PROMPTS.md
AI_NOTES.md
```

Структура может отличаться, если у вас есть более удобный и понятный вариант.